

C++ Programming: Systems And Programming Concepts

Chapter 2 Advanced String Functions

Advanced string functions

Recap:

- Some of the basic string functions learnt:
 - length, size, empty, at, insert, replace, append, copy(=), concatenate(+), compare(==, !=, >, <), substr, append, find, erase, getline, etc.

Advanced string functions

1. to_string(int)

- Converts numbers to strings

- Example:

```
string numStr = to_string(123);
```

2. stoi()

- Converts string to integer

- Example:

```
int num = stoi("123");
```

- stod() = string to double, stof() = string to float

Advanced string functions

3. `find_first_of(chars)`, `find_last_of(chars)`
 - Find the first/last position of any character from a string
 - Example:

```
string str = "Hello world";  
int pos = str.find_first_of("aeiou"); // 1
```
4. `find_first_not_of(chars)`, `find_last_not_of(chars)`
 - Finds the first/last character not in a string.

Advanced string functions

5. stringstream

- Splits strings or converts between string and numbers. Need to add **sstream** header file

- Example:

```
string text = "10 25.78 Hello";
```

```
int n; float m; string str;
```

```
stringstream ss(text);
```

```
ss >> n >> m >> str;
```

```
cout << n << " " << m << " " << str;
```

```
// Output: 10 25.78 Hello
```

Advanced string functions

5. stringstream

- Example:

```
int num = 45;  
stringstream ss;  
ss << num; // read 45 into string  
cout << ss.str(); // Output: 45
```

Advanced string functions

6. transform()

- Change each character in a string like uppercase, lowercase, applied with any rule
- `#include <algorithm> // add header file`
- *Syntax:*

transform (begin, end, destination, function);

begin, end: range to apply the function

destination: where to store the result

function: what to apply (e.g. `::toupper`, `::tolower`)

Advanced string functions

6. transform()

- Example:

```
string text1 = "Hello world!", text2 = "    ";  
transform(text1.begin(), text1.end(),  
          text1.begin(), ::toupper);  
cout << text1 << endl; // HELLO WORLD!
```

```
transform(text1.begin()+6, text1.end(),  
          text1.begin()+6, ::tolower);  
cout << text1 << endl; // HELLO world!
```


Advanced string functions

6. transform()

- Example (continued):

```
string text1 = "Hello world!", text2 = "    ";  
transform(text1.begin()+1, text1.end(),  
          text2.begin(), ::tolower);  
cout << text2 << endl; // ello
```

Advanced string functions

7. regex()

- Regular expression (regex) is a tool for pattern matching within strings.
- Can define search pattern using a special syntax and use the pattern to find, validate or manipulate text.
- `#include <regex> // add header file`

Advanced string functions

7. `regex( )` – the common functions

Function	Purpose
<code>regex_match</code>	Checks if the whole string matches the pattern
<code>regex_search</code>	Finds if any part matches
<code>regex_replace</code>	Replaces matching parts
<code>smatch</code>	Stores match results from search

Advanced string functions

7. regex() – some common regex patterns:

Pattern	Meaning
<code>\\d</code>	Digit (0–9)
<code>\\w</code>	Word character (letters, digits, _)
<code>.</code>	Any character
<code>*</code>	0 or more times
<code>+</code>	1 or more times
<code>^</code>	Start of string
<code>\$</code>	End of string

Advanced string functions

7. regex() - Examples

a) regex_search – find a pattern

```
string text = "My unit number is A-1-25";
```

```
regex pattern1("\\d+"); // \\d+ = one or more digits
```

```
if(regex_search(text, pattern1))
```

```
    cout << "Number found!";
```

```
else
```

```
    cout << "No number found!";
```



Advanced string functions

7. regex() - Examples

b) regex_match – match the whole string

```
string text = "abc123";
```

```
// Match lower case letters + digits
```

```
regex pattern2("[a-z]+\\d+");
```

```
if(regex_match(text, pattern2))
```

```
    cout << "Full match!";
```

```
else
```

```
cout << "No match!";
```



Advanced string functions

7. regex() - Examples

c) regex_replace – replace matched patterns

```
string pwd = "abc123";
```

```
// Match lower case letters + digits
```

```
regex pattern3("[a-z]+\d+");
```

```
string result = regex_replace(pwd,pattern3,"***");
```

```
cout << "Password entered" << result;
```

```
cout << "is correct!";
```

```
// Output: Password entered *** is correct!
```

Advanced string functions

7. regex() - Examples

d) regex_search – get matched parts

```
string text = "David is 18 years old";  
regex twoPatterns("(\\w+) is (\\d+) years old");  
smatch match;  
if(regex_search(text, match, twoPatterns)) {  
    cout << match[0] << endl;  
    cout << "Name is " << match[1] << endl;  
    cout << "Age is " << match[2] <<  
}
```

```
David is 18 years old  
Name is David  
Age is 18
```


Advanced string functions

8. `format( )`

- `#include <format>` // add header file

Use	Format Code Example
Insert variables	<code>"Hello, {}"</code>
Format floats	<code>"{: .2f}"</code> → 2 decimal places
Add leading zeros	<code>"{:05}"</code> → 00042
Align text	<code>"{:>6}"</code> or <code>"{:<6}"</code>
Multiple variables	<code>"{} is {} years old"</code>

Advanced string functions

8. `format( )`

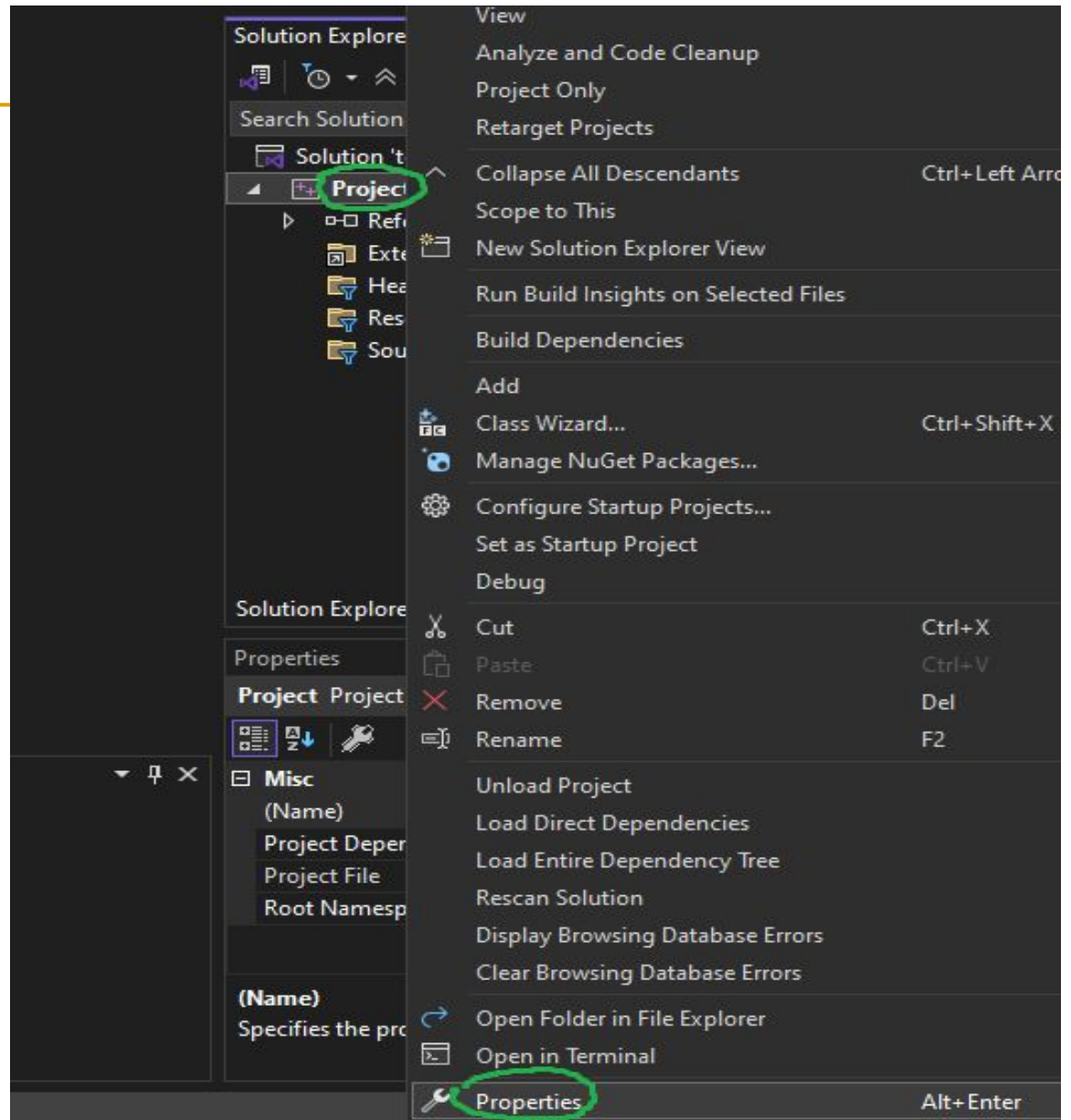
- enable ISO C++20 standard versioning

- Note:

C++ versions, often referred to as standards, include C++98, C++03, C++11, C++14, C++17, C++20, and the upcoming C++23. Each version introduces new features, language enhancements, and library additions.

Advanced string functions

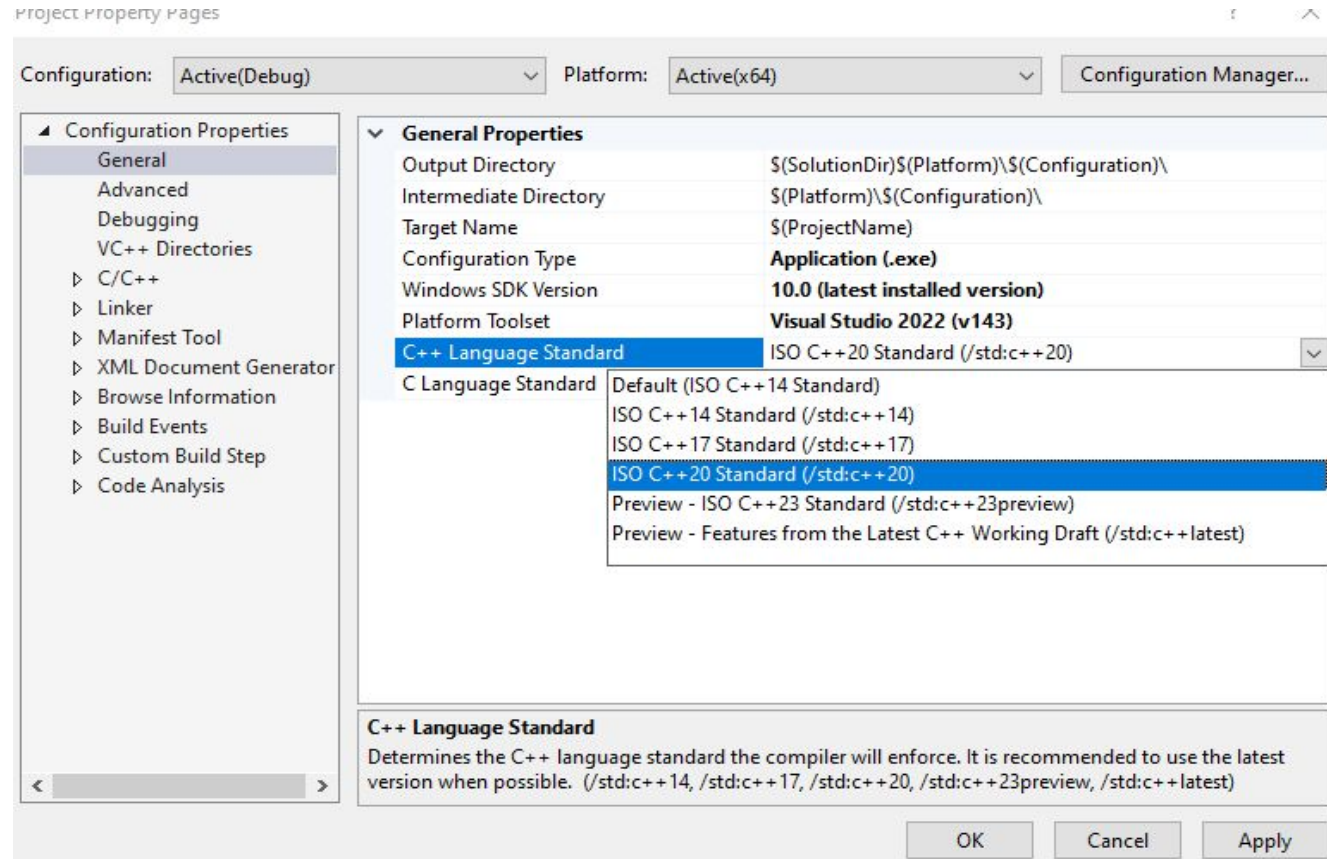
8. `format( )`
- enable ISO C++20 standard versioning



Advanced string functions

8. `format( )`

- enable ISO C++20 standard versioning



Advanced string functions

8. `format( )`

- Text and variables insertion
- Example:

```
string name = "Lionel", message1, message2;  
int age = 20;  
message1 = format("Hello, {} !", name);  
message2 = format("{} is {} years old", name,  
age);  
cout << message1 << message2;  
// Output: Hello Lionel ! Lionel is 20 years old
```

Advanced string functions

8. `format( )`

- Numeric values insertion

- Example:

```
    string result;  
    int num = 5;  
    const double pi = 3.14159;  
    result = format("{} x {:.2f } = {:.2f }",  
                   num, pi, num*pi);  
    cout << result;  
// Output: 5 * 3.14 = 15.71
```

Advanced string functions

8. format()

- Leading zeros and alignment
- Example:

```
int num = 68;
```

```
cout << format("Zero padded: {:05}\n", num);
```

```
cout << format("Right aligned: {:>6}\n", num);
```

```
cout << format("Left aligned : {:<6}\n", num);
```

```
/* Output: Zero padded : 00068
```

```
Right aligned:      68
```

```
Left aligned : 68      */
```

Advanced string functions

8. format()

- Leading zeros and alignment
- Example:

```
int num = 68;
```

```
cout << format("Zero padded: {:05}\n", num);
```

```
cout << format("Right aligned: {:>6}\n", num);
```

```
cout << format("Left aligned : {:<6}\n", num);
```

```
/* Output: Zero padded : 00068
```

```
Right aligned:      68
```

```
Left aligned : 68      */
```


Advanced string functions - Exercise

Find the areas of the following items which are circles of different radius.

Create and initialize the 2 vector arrays for these values and use string functions to format and tabulate the result as below:-

Items	Radius	Area
coin	0.5	$0.79cm^2$
ring	0.7	$1.54cm^2$
wheel	21.6	$1465.74cm^2$
:	:	:

Note: area of circle = πr^2 ($\pi = 3.14159$)

Advanced string functions - Exercise
